AI Software Engineering

Lezione 01 - Dall’intento al software verificato

Domain -> Requirements -> Spec -> Test -> Agent -> Verify

Obiettivo della lezione

Capire come cambia il ruolo dello sviluppatore quando l’implementazione può essere delegata a un agente: il valore si sposta dalla sola produzione di codice verso comprensione del dominio, esplicitazione dell’intento e verifica del risultato.

Durata consigliata: 35-45 minuti

1. Il cambio di paradigma

Tesi di partenza: il problema principale non è più soltanto scrivere codice. È trasformare un'intenzione in qualcosa di sufficientemente preciso da poter essere implementato e verificato.

> **Perché questo conta con gli agenti**
>
> Un agente è molto bravo a produrre codice plausibile. Se però il requisito è incompleto, tenderà a riempire i vuoti con assunzioni. Il rischio non è solo "codice sbagliato": è codice ben scritto che implementa decisioni che nessuno aveva realmente preso.

Un esempio

Immagina di chiedere: "Implementa un sistema che assegni gli interventi ai tecnici disponibili."

Prima ancora di parlare di framework o database, emergono domande di dominio:

- Che cosa significa "disponibile"? Libero in quel momento, reperibile, nella zona corretta o tutte queste cose?
- Che cosa significa "tecnico migliore"? Più vicino, meno carico, più qualificato o prioritario per contratto?
- Che cosa succede se nessuno è eleggibile?
- Chi può cambiare manualmente un'assegnazione automatica?

Principio: l'AI può proporre soluzioni, ma non deve inventare silenziosamente il problema.

2. Problem space e solution space

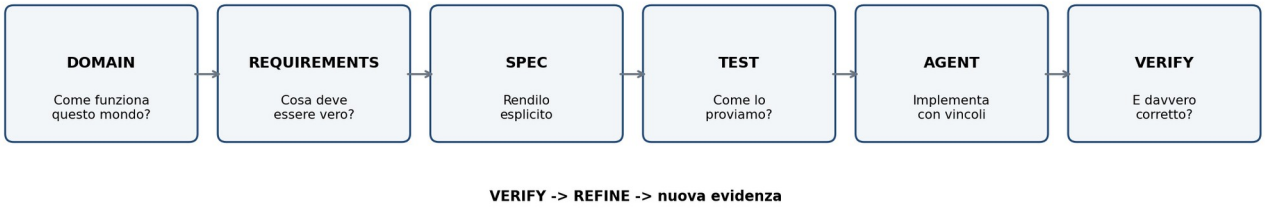
Una disciplina centrale del corso sarà separare deliberatamente il problema dalla soluzione tecnica.

PROBLEM SPACE	SOLUTION SPACE
• Che cosa deve succedere?	• Linguaggio / framework
• Perché è necessario?	• Database e struttura del codice
• Quali regole e invarianti esistono?	• API, eventi, code
• Quali termini sono ambigui?	• Pattern architetturali
• Quali decisioni sono ancora UNKNOWN?	• Implementazione concreta

> **Regola pratica**
>
> Finché non capisci bene il problem space, evita di saltare subito a "creiamo una tabella", "usiamo Redis", "facciamo un endpoint". Le tecnologie sono decisioni del solution space.

3. La spina dorsale: Domain -> Requirements -> Spec -> Test -> Agent -> Verify

Questa sequenza sarà la mappa del percorso di studio. Non è un processo rigido, ma un modello mentale per sapere quale problema stiamo risolvendo in ogni fase.



Fase	Domanda principale	Output tipico
Domain	Come funziona questo mondo?	Concetti, regole, linguaggio condiviso
Requirements	Cosa deve essere vero?	Comportamenti osservabili, vincoli, NFR
Spec	Come rendiamo l'intento esplicito?	Decisioni e criteri sufficientemente precisi
Test	Come possiamo dimostrarlo?	Acceptance test, unit/integration test, proprietà
Agent	Come delegare senza perdere controllo?	Piano, task, implementazione
Verify	È davvero ciò che volevamo?	Confronto spec/test/code + review/evals

4. Domain: capire il mondo prima del codice

Il dominio non è Laravel, Django, FastAPI, Node o PostgreSQL. Il dominio è il problema reale che il software rappresenta e le regole che lo governano.

Nel nostro caso di studio, alcuni termini del dominio sono:

- Intervento
- Tecnico
- Competenza
- Disponibilità
- Zona
- Priorità

- Assegnazione

Esempi di regole di dominio:

- Un intervento richiede almeno una competenza.
- Un tecnico può possedere più competenze.
- Un tecnico non può essere assegnato a due interventi incompatibili nello stesso intervallo temporale.

DDD in una frase

Domain-Driven Design parte dal modello del dominio e da un linguaggio condiviso. Nelle prossime lezioni vedremo perché questo diventa prezioso quando dobbiamo fornire a un agente una rappresentazione coerente del problema.

5. Requirements e Specification

Dal requisito vago al requisito osservabile

Debole: “Il sistema deve assegnare velocemente un tecnico.”

Più utile:

- Quando viene creato un intervento urgente, il sistema individua i tecnici disponibili nella fascia richiesta.
- Il tecnico deve operare nella zona dell’intervento.
- Il tecnico deve possedere la competenza richiesta.
- Tra gli eleggibili, viene applicato un criterio di selezione esplicito e verificabile.

Una possibile specifica

Caso di studio

Progettare un sistema che assegni automaticamente un intervento di manutenzione urgente a un tecnico disponibile e qualificato.

Precondizioni	L’intervento è URGENT, ha una zona e almeno una competenza richiesta.
Tecnico eleggibile	È disponibile; opera nella zona; possiede la competenza richiesta.
Selezione	Tra gli eleggibili si applica il criterio deciso dal dominio (es. minor carico giornaliero).
Tie-breaker	A parità si applica una seconda regola esplicita (es. distanza).

Nessun tecnico	L’intervento resta UNASSIGNED e viene generato un evento di mancata assegnazione.

6. Invarianti: il ponte tra dominio e test

Un’invariante è una condizione che deve rimanere vera. È una delle forme più potenti di informazione perché collega dominio, specifica e verifica.

Regola di dominio del caso di studio

Un tecnico privo della competenza richiesta dall’intervento non è idoneo all’assegnazione.

Domain rule -> Requirement -> Specification -> Executable test

Pseudotest:

```
technician = Technician(skills={"plumbing"})
job = Job(required_skill="electrical")

result = assign(job, technician)

assert result.is_rejected()
```

Attenzione

Test verdi dimostrano che l’implementazione soddisfa ciò che abbiamo formalizzato. Non dimostrano, da soli, che abbiamo capito correttamente il problema o scelto il requisito giusto. Per questo Domain Discovery e Requirements Engineering vengono prima di TDD.

7. Agent: implementatore, non autorità

Confronta questi due incarichi:

Input debole	Input vincolato
“Implementa l’assegnazione dei tecnici.”	“Implementa la feature descritta nella spec. Non modificare i requisiti. Individua ambiguità, prepara le prove, implementa il minimo necessario e riesegui le verifiche.”

Verify non significa “i test passano”

Una fase di verifica robusta può includere:

- spec <-> implementation
- spec <-> tests

- tests <-> requirements
- static analysis e type checking
- integration/contract tests
- security checks
- architecture constraints
- human review e evals

8. Ponte operativo: il workflow di Matt Pocock

Le skill di Pocock ci serviranno come laboratorio concreto. Il corso, però, studierà anche i principi sottostanti, così da capire quando usare, adattare o sostituire il workflow.



Le skill sono il laboratorio; DDD, requirements, SDD, TDD e verification spiegano il perche.

La lettura concettuale del workflow è:

- grill-with-docs: far emergere ambiguità, convergere sul linguaggio e registrare contesto/decisioni;
- to-spec: trasformare decisioni già prese in una specifica, senza usare la spec per “inventare” il prodotto;
- to-tickets: decomporre il lavoro in vertical slice sufficientemente piccoli e agent-ready;
- implement + TDD: eseguire iterativamente mantenendo feedback rapido;
- code-review: verificare conformità a specifica e standard.

Obiettivo del corso

Non imparare a memoria cinque comandi. Imparare perché esistono: domain discovery, requirements, SDD, TDD, context engineering e verification. Se domani cambiano le skill, il metodo resta comprensibile e adattabile.

9. Laboratorio - 15 minuti, niente codice

Scenario: “Vorrei una piattaforma che assegni automaticamente le richieste di assistenza ai tecnici. Deve scegliere il tecnico migliore e avvisarlo. Se non risponde, deve trovarne un altro.”

Produci soltanto questi artefatti:

1. 5 termini del dominio che devono essere chiariti.
2. 5 domande da fare al cliente prima di progettare la soluzione.
3. 3 requisiti osservabili senza dettagli tecnologici.
4. 2 possibili invarianti.
5. 3 scenari: happy path, edge case e failure case.
6. 1 test in pseudocodice derivato da una regola.
7. 1 prompt per un coding agent che lo autorizzi a implementare solo ciò che è già stato deciso.

Regola dell'esercizio

Non puoi inventare le risposte alle tue stesse domande. Se non sai cosa significa "tecnico migliore", scrivi UNRESOLVED: criterio di selezione del tecnico.

KNOWN | ASSUMED | UNKNOWN | DECIDED

10. Takeaway della Lezione 1

1. Separare il problema dalla soluzione.
2. Esplicitare ciò che un agente altrimenti assumerebbe.
3. Trattare invarianti e comportamenti osservabili come ponte verso la verifica.
4. Delegare l'implementazione senza delegare silenziosamente le decisioni di dominio.
5. Considerare VERIFY come fase autonoma, non come semplice "tutti i test verdi".
6. Usare le skill operative (come quelle di Pocock) come strumenti dentro un metodo di progettazione più ampio.

Prossima lezione

Domain Discovery: come capire un dominio prima di modellarlo. Collegheremo DDD, Ubiquitous Language e tecniche di discovery al modo in cui grill-with-docs cerca di ottenere una comprensione condivisa con l'agente.

Letture e riferimenti

The Pragmatic Programmer - 20th Anniversary Edition: https://pragprog.com/titles/tpp20/

GitHub Spec Kit - documentazione: https://github.github.com/spec-kit/

Agentic Software Engineering - libro open access: https://book.agentic-swe.dev/

Martin Fowler - Domain Driven Design (bliki): https://martinfowler.com/bliki/DomainDrivenDesign.html

Matt Pocock skills - repository/documentazione: https://github.com/mattpocock/skills